INFORME DE LABORATORIO #2

Expresiones Regulares con JFLAP

Antecedente	Información
Asignatura	Teoría de la Computación INFO1148
Carrera	Ingeniería Civil en Informática
Docente	M. Lévano
Integrantes	Nataly Huaiquinao – Diego Sanhueza
Fecha	11 de octubre de 2026

1. Introducción

En este laboratorio se trabajó la equivalencia entre expresiones regulares (ER), autómatas finitos no deterministas (AFN) y autómatas finitos deterministas (AFD) utilizando el programa JFLAP. En cada una de las preguntas se elabora el diseño o la interpretación de las expresiones regulares, se convierten en autómatas y se validan mediante casos de prueba. Se utilizó la sintaxis de JFLAP: `+` para la unión y `*` para la cerradura de Kleene.

2. Desarrollo

2.1. Pregunta 1 – Interpretación y validación de una expresión regular

a) Lenguaje representado

Alfabeto $\Sigma = \{0, 1\}$ y expresión regular `(0+1)*01`.

La expresión genera cualquier cadena binaria con `(0+1)*` (incluida la vacía) y luego obliga a que la cadena termine con un 0 seguido de un 1. Es decir, representa todas las cadenas binarias que terminan en 01, sin importar lo que haya antes:

$$L_1 = \{w \in \{0,1\}^* \mid w \text{ termina en } 01\}.$$

Se ingresó la expresión en *Regular Expression* y se utilizó *Convert* → *Convert to NFA*. Archivo: `P1_AFN.jff`.

b) Resultados obtenidos

Cadena	Predicción	Resultado JFLAP	¿Coinciden?
01	Aceptada	Aceptada	Sí
101	Aceptada	Aceptada	Sí
1101	Aceptada	Aceptada	Sí
0001	Aceptada	Aceptada	Sí
10	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
0100	Rechazada	Rechazada	Sí
10101	Aceptada	Aceptada	Sí

Tabla 1: Predicciones frente a resultados en JFLAP (Pregunta 1).

c) Evidencia en JFLAP

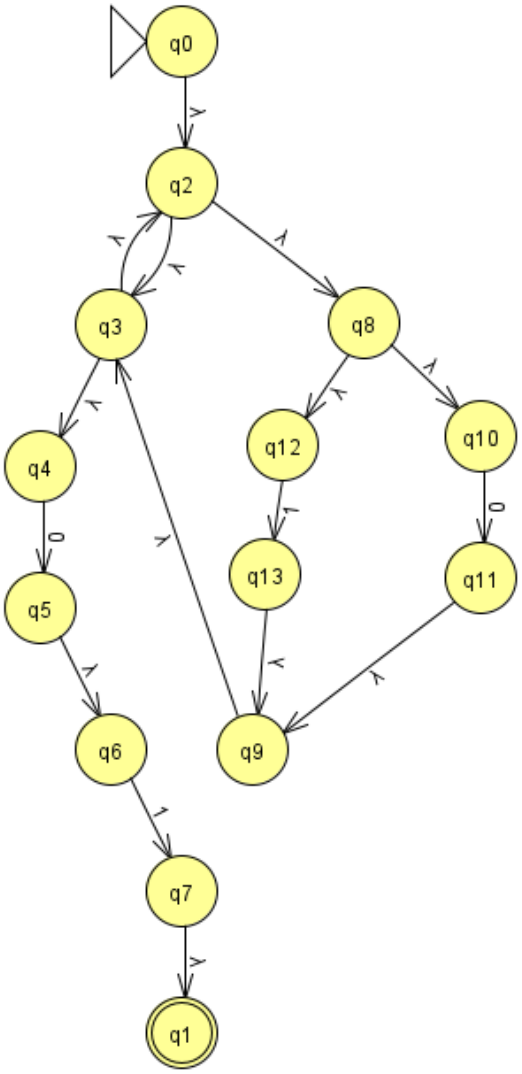


Figura 1: AFN obtenido en JFLAP para `(0+1)*01`.

d) Análisis

Las 8 predicciones coincidieron con los resultados obtenidos con JFLAP, de las cuales 5 cadenas fueron aceptadas y 3 fueron rechazadas. El AFN tiene muchas transiciones λ, esto se debe a que JFLAP descompone la ER por partes: la cerradura `(0+1)*` se transforma en un ciclo entre q2 y q3 (con las ramas que leen 0 o 1) y luego se encadenan las transiciones que leen el 0 (q4 $\rightarrow$ q5) y el 1 (q6 $\rightarrow$ q7) antes de llegar al estado final q1.

- `1101` **es aceptada:** la cadena se puede dividir como $11 \cdot 0 \cdot 1$. El prefijo 11 se consume dando dos vueltas por el ciclo de `(0+1)*`; luego el AFN sale del ciclo por λ (q3 $\rightarrow$ q4), lee el 0 (q4 $\rightarrow$ q5), lee el 1 (q6 $\rightarrow$ q7) y llega por λ al estado final q1. Basta con que exista un camino que termine en un estado final para que un AFN acepte la cadena.
- `0100` **es rechazada:** sus dos últimos símbolos son 00 y no 01. La única forma de llegar a q1 es leyendo un 1 en la transición q6 $\rightarrow$ q7, y la cadena termina en 0. Por eso, ningún camino que consuma toda la cadena termina en el estado final. Aunque 0100 contiene 01, esa subcadena no está al final.

2.2. Pregunta 2 – Diseño de una expresión regular

a) Expresión regular propuesta

Lenguaje: $L_2 = \{w \in \{a, b\}^* \mid w \text{ comienza con } a \text{ y termina con } b\}$.

Expresión regular: `a(a+b)*b`

b) Explicación

La expresión se diseñó sin usar JFLAP, como la concatenación de tres partes:

- `a` **(inicio):** es el primer símbolo de la expresión, por lo que obliga a que toda cadena comience con a.
- `(a+b)*` **(secuencia intermedia):** permite cualquier combinación de a y b, de cualquier largo, incluso vacía.
- `b` **(final):** es el último símbolo de la expresión, por lo que obliga a que toda cadena termine con b.

La cadena de longitud mínima es ab, que se obtiene cuando `(a+b)*` genera la cadena vacía. Archivo: `P2_AFN.jff`.

c) Casos de prueba

Cadena	Resultado esperado	Resultado JFLAP
`ab`	Aceptada	Aceptada
`aab`	Aceptada	Aceptada
`abb`	Aceptada	Aceptada
`abab`	Aceptada	Aceptada
`aaabbb`	Aceptada	Aceptada
`a`	Rechazada	Rechazada
`b`	Rechazada	Rechazada
`ba`	Rechazada	Rechazada
`abba`	Rechazada	Rechazada
`bab`	Rechazada	Rechazada
λ (vacía)	Rechazada	Rechazada

Tabla 2: Casos de prueba de la Pregunta 2.

d) Evidencia en JFLAP

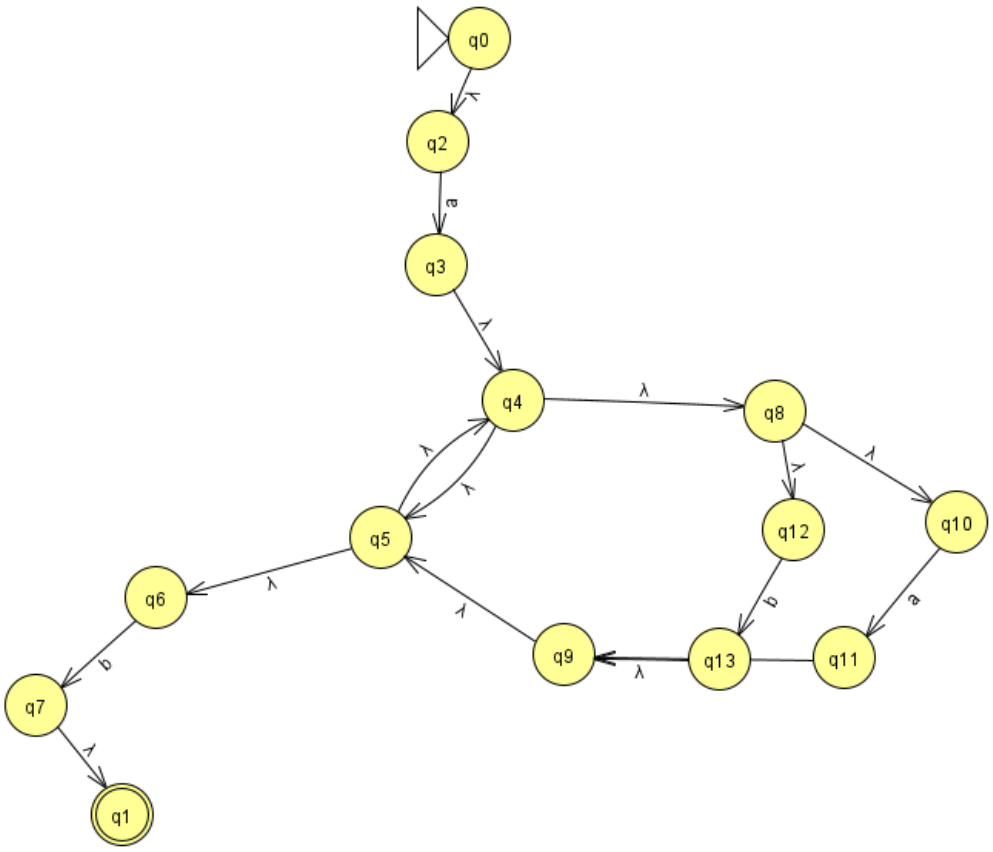


Figura 2: AFN obtenido en JFLAP para `a(a+b)*b`.

e) Análisis

Probamos 11 cadenas, donde 5 fueron aceptadas y 6 rechazadas. Todos los resultados que obtuvimos en JFLAP coincidieron con las predicciones.
Incluimos la cadena de longitud mínima (ab) y varios casos límite: la cadena vacía λ y las cadenas de un solo símbolo (a y b), que no pueden comenzar con a y terminar con b al mismo tiempo. Las cadenas ba, $abba$ y bab muestran que basta con no cumplir una de las dos condiciones para que la cadena sea rechazada.
El AFN que obtuvimos refleja la estructura de la expresión regular. La transición q2 → q3 lee la a obligatoria del inicio. El ciclo entre q4 y q5, con sus dos ramas (q10 → q11 lee a y q12 → q13 lee b), corresponde a `(a+b)*` y permite cualquier secuencia intermedia. Finalmente, la transición q6 → q7 lee la b obligatoria del final antes de llegar por λ al estado final q1. Por esto, una cadena solo llega al estado final si comienza con a y termina con b.

2.3. Pregunta 3 – Conversión ER → AFN → AFD

a) Expresión regular propuesta

Lenguaje: $L_3 = \{w \in \{0,1\}^* \mid w \text{ contiene la subcadena } 101\}$.

ER = `(0+1)*101(0+1)*`

Los prefijos y sufijos arbitrarios permiten que 101 aparezca en cualquier posición. Archivos: `P3_AFN.jff` y `P3_AFD.jff`.

b) Características de los autómatas

Característica	AFN	AFD
Número de estados	24	9
Número de transiciones	29	18
Transiciones λ	Sí	No
Determinista	No	Sí

Tabla 3: Características de los autómatas de la Pregunta 3.

c) Pruebas realizadas

Cadena	Resultado AFN	Resultado AFD	¿Coinciden?
`101`	Aceptada	Aceptada	Sí
`0101`	Aceptada	Aceptada	Sí
`1010`	Aceptada	Aceptada	Sí
`11011`	Aceptada	Aceptada	Sí
`000`	Rechazada	Rechazada	Sí
`111`	Rechazada	Rechazada	Sí
`1001`	Rechazada	Rechazada	Sí
`1101010`	Aceptada	Aceptada	Sí

Tabla 4: Comparación entre AFN y AFD (Pregunta 3).

d) Evidencias

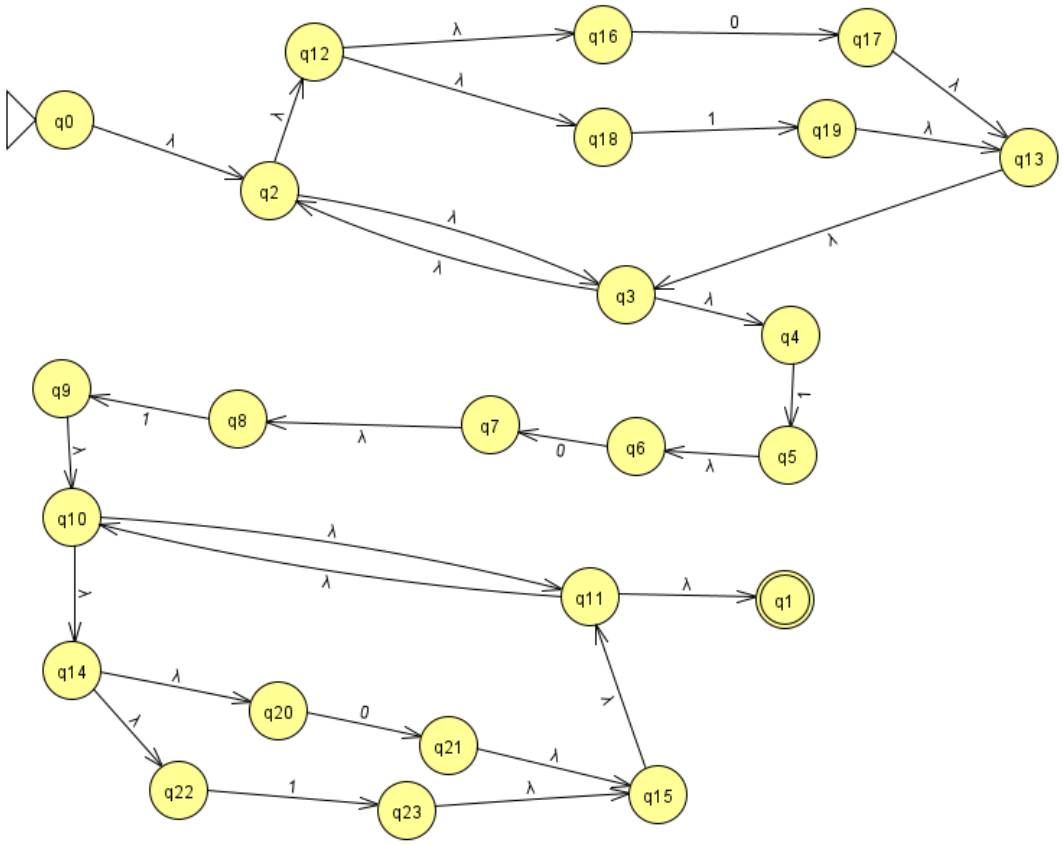


Figura 3: AFN obtenido desde la ER.

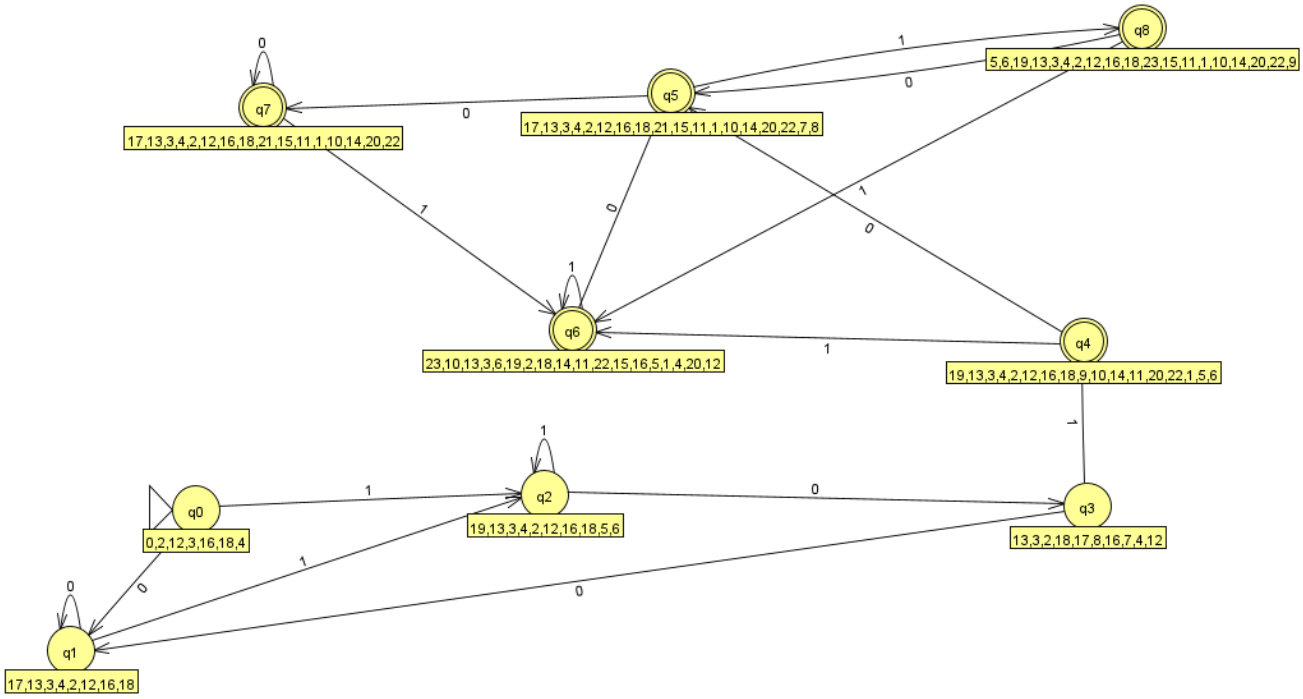


Figura 4: AFD obtenido desde el AFN.

e) Análisis

¿El AFN y el AFD reconocen lenguajes diferentes?
Basándonos en los resultados obtenidos tanto en el AFN como en el AFD, podemos concluir que, a pesar de tener estructuras diferentes, llegan al mismo resultado, lo que evidencia que reconocen el mismo lenguaje.
Esto se explica por la construcción de subconjuntos que realiza JFLAP: cada estado del AFD representa el conjunto de estados en que puede encontrarse el AFN después de leer un mismo prefijo, y es final si ese conjunto contiene el estado final del AFN. Por eso el AFD acepta una cadena si y solo si algún camino del AFN la acepta. Las diferencias son solo estructurales: el AFN tiene 24 estados y transiciones λ, mientras que el AFD tiene 9 estados y exactamente una transición por símbolo en cada estado.

2.4. Pregunta 4 – Diseño y validación

a) Expresión regular propuesta

Lenguaje: cadenas sobre $\Sigma = \{a, b\}$ que contienen al menos una a y terminan en bb.

ER = `(a+b)*a(a+b)*bb`

b) Justificación

La estrategia que utilizamos para construir la expresión regular fue establecer primero una combinación de letras, en este caso a y b, escrita como `(a+b)`. Para que esa elección se pueda repetir cero o más veces, añadimos el `*`, que representa cualquier cadena posible, incluida la vacía. A continuación se dejó la `a` como condición para que haya al menos una a. Para que se cumpla la otra condición, volvimos a permitir cualquier combinación de letras con `(a+b)*`, pero obligando a que la cadena termine en `bb`. Archivos: `P4_AFN.jff` y `P4_AFD.jff`.

c) Casos de prueba

Cadena	Resultado esperado	AFN	AFD
`aabb`	Aceptada	Aceptada	Aceptada
`babbb`	Aceptada	Aceptada	Aceptada
`ababbb`	Aceptada	Aceptada	Aceptada
`baabb`	Aceptada	Aceptada	Aceptada
`abbb`	Aceptada	Aceptada	Aceptada
`aaa`	Rechazada	Rechazada	Rechazada
`aba`	Rechazada	Rechazada	Rechazada
`ab`	Rechazada	Rechazada	Rechazada
`baaa`	Rechazada	Rechazada	Rechazada
`bba`	Rechazada	Rechazada	Rechazada
`abb`	Aceptada	Aceptada	Aceptada
`bb`	Rechazada	Rechazada	Rechazada

Tabla 5: Casos de prueba de la Pregunta 4 (5 aceptadas, 5 rechazadas y 2 casos límite).

d) Evidencias

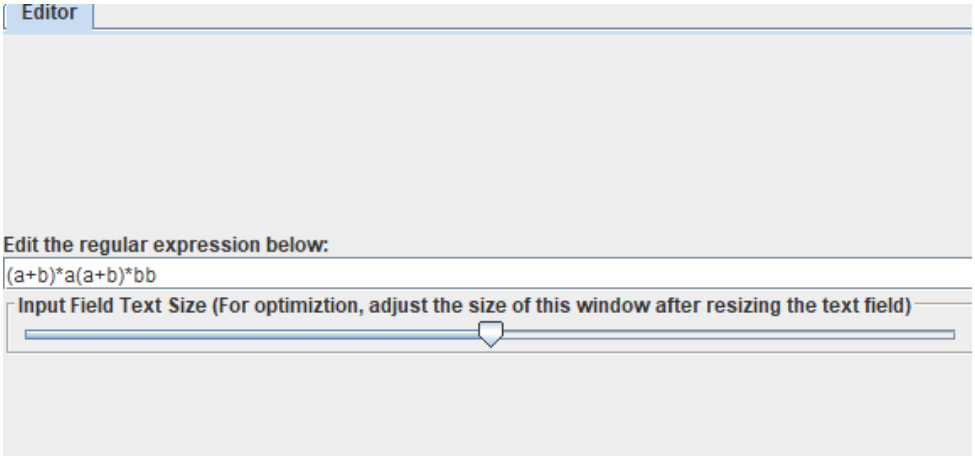


Figura 5: Expresión regular ingresada en JFLAP.

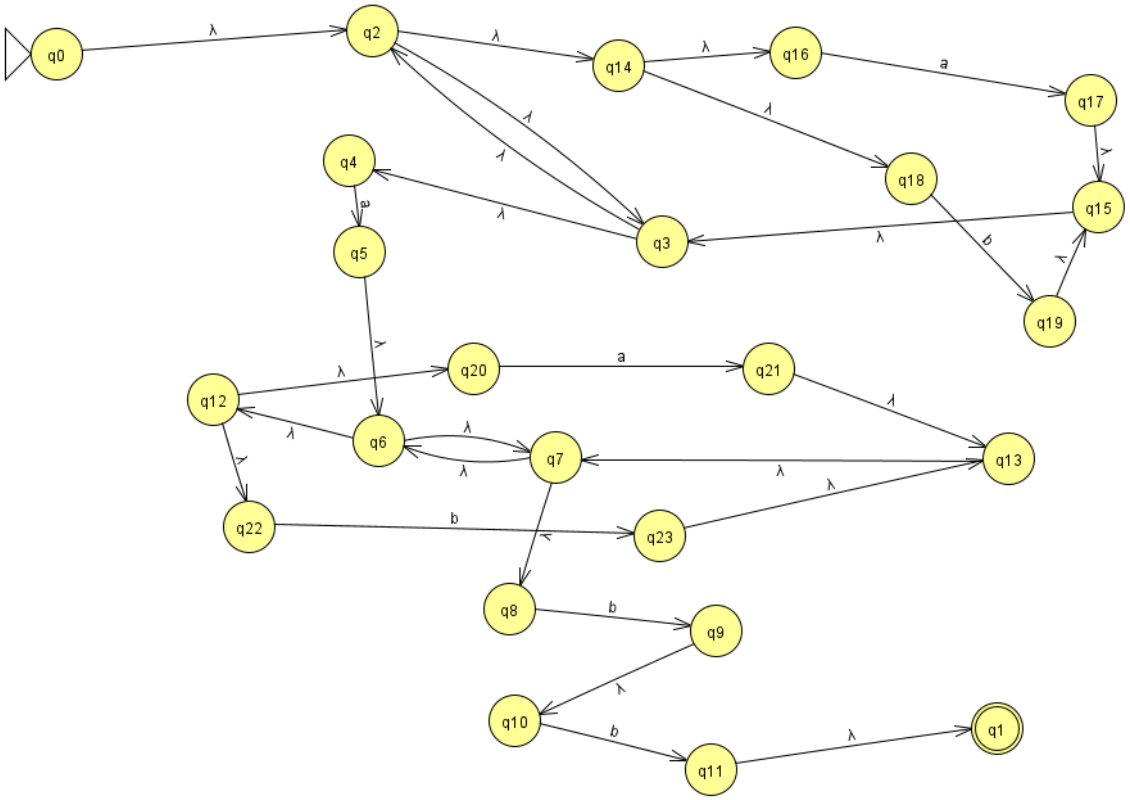


Figura 6: AFN generado.

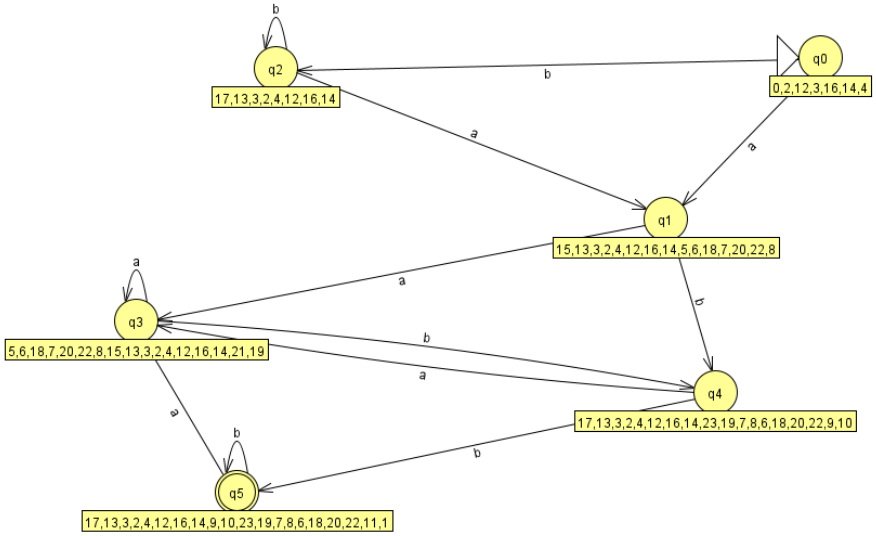


Figura 7: AFD resultante.

e) Reflexión

La expresión regular, el AFN y el AFD no poseen distinta capacidad de reconocimiento; en cambio, los tres son formas diferentes de representar una misma clase de lenguajes. La ER lo describe mediante una expresión; el AFN lo reconoce de forma no determinista, lo que le permite seguir varios caminos y usar transiciones λ; y el AFD lo reconoce de forma determinista. La principal diferencia está en cómo se representan, puesto que toda ER puede convertirse en un AFN y este, mediante la construcción por subconjuntos, en un AFD equivalente que acepta exactamente las mismas cadenas. Lo comprobamos probando las cadenas, que dieron el mismo resultado en los tres; por lo tanto, ER, AFN y AFD representan un mismo lenguaje.

3. Conclusiones

Durante este laboratorio pudimos aprender cómo se relacionan las ER, los AFN y los AFD, comprobando que representan el mismo lenguaje regular y cumpliendo con los resultados de diseñar ER, convertirlas y validarlas con JFLAP.

Como se pudo comprobar en cada una de las preguntas de este laboratorio, las predicciones coincidieron con los resultados obtenidos en JFLAP. Las cadenas dieron el mismo resultado en la ER, el AFN y el AFD.

4. Archivos adjuntos

Archivos `.jff` entregados junto con el informe:

- `P1_AFN.jff`
- `P2_AFN.jff`

- `P3_AFN.jff`
- `P3_AFD.jff`
- `P4_AFN.jff`
- `P4_AFD.jff`

Enlace al repositorio (GitHub/Drive) con el informe en PDF y los archivos `.jff`: [https://github.com/DiegoSanhueza13/laboratorio-2].